Défi d'adaptation : un cas nouveau, traité seul

Situation

Datalys vous confie un petit cas, jamais vu : 600 points décrits par **deux mesures**, à classer en **deux classes**. Le client demande : « un réseau de neurones fait-il mieux qu'une méthode simple, et peut-on le garantir ? » Vous répondez seul·e, avec des chiffres. Aucun pas à pas ne vous est donné : vous réutilisez ce que vous avez appris dans les TP.

Les données

```
from sklearn.datasets import make_moons
X, y = make_moons(n_samples=600, noise=0.35, random_state=42)
```

Ces données sont générées : aucun fichier à télécharger. Les deux classes ne se séparent pas par une droite.

Ce que vous devez faire

1. **Découper** les données en apprentissage et test, en gardant les proportions des classes.
2. **Mettre à l'échelle** les mesures, sans fuite : l'ajustement se fait sur l'apprentissage seulement.
3. **Établir une référence non neuronale** (une régression logistique) et la mesurer sur le test.
4. **Entraîner un réseau** avec Keras, avec un arrêt précoce sur une validation tirée de l'apprentissage.
5. **Comparer deux réglages** du réseau (par exemple la taille de la couche cachée, le taux d'apprentissage, le dropout), chacun sur **trois graines**.
6. **Mesurer** : accuracy, F1 macro et matrice de confusion, pour la référence et pour le réseau.
7. **Calculer la marge de bruit** liée à la taille du test et comparer l'écart entre le réseau et la référence à cette marge.
8. **Conclure** en une demi-page : le réseau est-il meilleur que la référence, de combien, et ce gain est-il plus grand que le bruit ? Si la réponse est non, dites-le.

Boîte à outils

Des noms à chercher dans la documentation, sans exemple :

Besoin	À chercher
Découper en gardant les proportions	`train_test_split`, argument `stratify`
Mise à l'échelle sans fuite	`StandardScaler`, `fit` puis `transform`
Méthode de référence	`LogisticRegression`
Réseau	`Sequential`, `Dense`, `Dropout`, `compile`, `fit`
Arrêt précoce	`EarlyStopping`, `restore_best_weights`
Reproductibilité	`set_random_seed`
Mesures	`accuracy_score`, `f1_score` (moyenne `macro`), `confusion_matrix`
Marge de bruit	formule du TP5 : 1,96 × racine de p(1−p)/n

Livrables

- Un notebook `defi_moons_<prenom>.ipynb` que vous créez, exécuté de haut en bas sans erreur.
- Une conclusion d'une demi-page, dans le notebook.